

PS 26168 — Fact-Check Findings + 6-Slide Content Blueprint

Part 1: What I verified, corrected, or couldn't confirm

1. The problem statement itself

- **Confirmed real and current for SIH 2026:** PS ID is **SIH26168**, titled "*AI-ML based Intelligent Dead Reckoning system for seamless navigation*," sponsored by ISRO, **Software** category. This is consistent across the official-style master catalogue and three independent scraped mirrors of sih.gov.in.
- **Minor discrepancy, unresolved:** the theme tag is inconsistent across sources — most list it as "**Miscellaneous**," one lists it as "**Smart Automation**." Doesn't affect your build; just don't be surprised if the portal shows a different theme label than your source-of-record.
- **Important limitation — I could not verify your specific numeric claims against the verbatim official text.** Despite extensive searching, every third-party archive I could reach gave me the PS number/title/org but not a reliably complete official problem body for 26168 specifically. One independent source (a YouTube walkthrough of the SIH 2026 launch) explicitly warns that the official portal listing for each PS is often just a title plus a short paragraph, with more elaborate specs sometimes added later or inferred by community sites. **This means I cannot confirm, from primary source, that:**
 - the exact "<10% drift" benchmark and the "5m/50m" and "100m/1km" examples are official portal language vs. reasonable inference by the AI report authors,
 - IO-VNBD is stated as *mandatory* (vs. suggested) in the literal PS text,
 - UKF is *literally* named as "suggested" and ESKF is a substitution choice, or
 - the FOG IMU / 200Hz example is really in the PS. **Action for you:** before you print these numbers on your title/feasibility slides, have one teammate pull up the literal PS text on sih.gov.in (search "26168") and screenshot it. If the benchmark numbers match what's in your source reports, great — use them confidently. If the portal text is vaguer, don't put a specific "<10%" figure in the deck as if ISRO mandated it; treat it as *your team's own target metric*, which is actually a stronger, more defensible framing for judges anyway ("we set ourselves a rigorous internal bar of <10% drift").

2. IO-VNBD dataset — accessible and structurally as described, with one correction

- **Confirmed live:** github.com/onyekpeu/IO-VNBD is a real, currently accessible public repo (18 stars, 6 forks), tied to a peer-reviewed *Data in Brief* (2021) paper by Onyekpe, Palade, Kanarachos & Szkolnik (Coventry University).
- **Confirmed structure:** two parallel streams — smartphone data ("S-" prefix, 10 Hz IMU + 1 Hz GNSS) and vehicle ECU/CAN data ("V-" prefix, 10 Hz, wheel speeds + steering + yaw rate), roughly 40h/1,300km vehicle-side and 58h/4,400km phone-side, ~98h/5,700km combined — this matches both source reports closely.
- **Correction:** the data was **not** collected "predominantly in the UK" in the narrow sense one report implies — the official data descriptor states collection happened on public roads in **the United Kingdom, Nigeria, and France**. This is actually good news for your generalization story (Slide 4): you can honestly say the dataset already spans multiple countries/road cultures, which slightly weakens (but doesn't eliminate) the "UK-trained model on Indian roads" overfitting risk you should still plan to test for.

3. Architecture sanity-check against current literature — your recommendation holds, with two corrections worth making in the deck

- **The core recommendation (AI regresses forward velocity → feeds a classical ESKF with Non-Holonomic Constraints → HMM/Viterbi map-matching) is still the most literature-supported "safe" choice for a first-time team.** Nothing has emerged since these reports were written that changes this. This is consistent with the published academic pattern (WhONet-style "AI as virtual odometer" architectures, and Brossard et al.'s AI-IMU Dead-Reckoning, both real, cited, code-available work).
- **Correction — TLIO domain-transfer risk is real, not paranoia.** I checked directly: TLIO (Liu et al., "Tight Learned Inertial Odometry") was trained and validated **only on pedestrian headset data**; its own abstract and paper text describe pedestrian motion exclusively. I found **no published vehicular validation** of TLIO's tightly-coupled displacement+covariance approach. This confirms the flag in your source material — avoid TLIO-style tightly-coupled learned displacement architectures as your primary path; the "safe hybrid" (AI-for-velocity + classical ESKF) sidesteps this risk entirely because it only asks the network to learn a much narrower, better-understood quantity (scalar forward speed), which *does* have vehicular precedent (WhONet family, all vehicle-tested).
- **A better-fit reference than TLIO exists and is underused in your source material:** *Brossard, Barrau & Bonnabel, "AI-IMU Dead-Reckoning" (2019)* — a neural-network-tuned EKF tested on the **KITTI vehicle dataset**, achieving ~1.1% translational error using only an IMU, with working open-source code. It's directly vehicular (unlike TLIO) and closer in spirit to your "AI assists, classical filter governs" philosophy. Worth citing on Slide 6 instead of/alongside TLIO.
- **Correction — Android NNAPI is deprecated.** Both source reports lean heavily on "ONNX Runtime NNAPI Execution Provider" / "LiteRT NNAPI delegate" for edge inference. As of **Android 15, NNAPI is deprecated** (confirmed via Android Open Source Project docs and Google's own LiteRT issue tracker). It still works for backward compatibility, but new work should target **LiteRT's CompiledModel API with**

GPU/XNNPACK delegates instead. Practically this barely matters for your case — a 1D-CNN/TCN velocity regressor is tiny and will run comfortably on CPU via XNNPACK even without NPU acceleration — but don't tell judges "we use NNAPI" as if it's the current best practice; say "LiteRT" or "ONNX Runtime CPU/XNNPACK," which is more accurate and future-proof.

4. Feasibility for your specific team

Given 2nd-year students, no prior ML/robotics project experience, building via Claude Code + Gemini Pro, phone-only scope:

Component	Real skill gap?	Why
TCN/1D-CNN training on IO-VNBD	Moderate. Standard supervised regression (window of IMU → scalar velocity) is one of the most beginner-tractable parts of this whole stack — it's ordinary PyTorch, well-documented, and IO-VNBD gives you a clean ground-truth target (CAN wheel speed) to train against.	Low math burden, high tooling support.
ESKF math (15-state, NHC pseudo-measurements)	High — the real bottleneck. Error-state formulations, quaternion error propagation, and NHC as a measurement update are genuinely unfamiliar territory for a team with no robotics background. This is where you'll burn the most hours and need the most AI-agent scaffolding (have Claude Code work from one of the reference implementations below line-by-line rather than deriving from scratch).	Requires understanding of covariance propagation, not just API calls.
Android sensor/foreground-service plumbing + NNAPI/LiteRT deployment	Moderate. Mechanically well-documented (Android docs are thorough), but doze-mode/wake-lock debugging is the kind of thing that eats a day if you haven't hit it before.	Tooling-heavy, not math-heavy.
HMM/Viterbi map matching against OSM	Moderate-to-low if you use Valhalla's meili service rather than writing your own HMM. Valhalla is a real, actively maintained open-source routing engine (github.com/valhalla/valhalla) with a documented map-matching module (meili) — treat this as "integrate a library," not "implement Viterbi from scratch."	Real risk is JNI/build integration time, not the algorithm itself.

Bottom line: the ESKF is your critical path and biggest single risk. Everything else is tractable in 8 weeks; budget disproportionate time (and a fallback to a simpler 2D linear KF, as your own source material already suggests for a compressed timeline) here.

5. Reference implementations (verified real, with working code)

(a) IMU/wheel-signal → velocity regression:

- github.com/onyekpeu/WhONet — real code from a published, peer-reviewed paper (Onyekpe et al., *Engineering Applications of AI*, 2021), same author group as IO-VNBD. **Caveat:** WhONet's input is wheel-speed (CAN "V-" data), not raw phone IMU — it's the closest conceptual template but you'll need to adapt it to take "S-" IMU windows as input instead, training against "V-" wheel speed as the label (exactly as Report 2 suggests).

(b) ESKF with non-holonomic constraints (Python/C++):

- github.com/jasleon/Vehicle-State-Estimation — real, documented ES-EKF fusing IMU+GNSS+LiDAR on CARLA simulator data, from a University of Toronto Coursera course. Good scaffold to adapt (swap LiDAR update for your NHC pseudo-measurement update).
- github.com/enginBozkurt/Error-State-Extended-Kalman-Filter — same lineage, similar starting point.
- github.com/mbrossar/ai-imu-dr — Brossard et al.'s AI-IMU Dead-Reckoning, vehicle-tested on KITTI, directly relevant as a "how do you responsibly blend a neural signal into an EKF for a car" reference.

(c) HMM map-matching against OSM:

- github.com/valhalla/valhalla — the meili service handles HMM/Viterbi map-matching against OSM natively; treat as integration work, not an algorithm you write yourselves.

Part 2: Slide-by-Slide Content Blueprint

Slide 1 — Title Page

- **Problem Statement ID:** SIH26168 (*verify theme label on portal — sources disagree; ID and title are solid*)
- **Problem Statement Title:** AI-ML based Intelligent Dead Reckoning system for seamless navigation
- **Theme:** Miscellaneous (*confirm on sih.gov.in — one mirror shows "Smart Automation"*)
- **PS Category:** Software

- **Team Name:** [your team name]
- *No diagram needed — this is a fixed-field cover slide.*

Slide 2 — Idea Title / Proposed Solution

Bullets:

- **Neural Virtual Odometer:** we turn the phone's IMU into an AI-powered "synthetic wheel-speed sensor," feeding a classical navigation filter instead of replacing it — the safe way to combine AI with physics.
- Aligns the phone's arbitrary position (dashboard, cup-holder, pocket) to the car's motion automatically — no setup step for the user.
- A small on-device neural network estimates the car's forward speed directly from vibration/motion patterns, even through potholes and engine noise.
- That AI-estimated speed feeds a proven aerospace-grade filter (Error-State Kalman Filter) that also uses the simple physical fact that cars don't skid sideways or fly.
- Final position is snapped onto real roads using open map data — so the dot never floats into a building or a field.
- **Innovation/uniqueness:** unlike naive "double integrate the accelerometer" approaches (which fail within seconds), we scope AI to one narrow, learnable job — speed estimation — and let 100+ years of proven navigation math do the rest. This is the same design philosophy used in real published vehicle-navigation research (not just theory).

Diagram to include: simple left-to-right block diagram: Phone IMU (accelerometer + gyroscope) → Auto-Alignment Engine → AI Velocity Estimator (TCN/1D-CNN) → Error-State Kalman Filter (+ GNSS when available + Non-Holonomic Constraints) → Map-Matching (snap to road) → Final Trajectory

Slide 3 — Technical Approach

Bullets:

- **Languages/frameworks:** Python + PyTorch (model training), ONNX Runtime / LiteRT (on-device inference), Kotlin/Java (Android app), C++ or Python (filter), Valhalla (offline map-matching).
- **Hardware:** standard Android smartphone only — no OBD-II, no external sensors, no special hardware.
- **Methodology (4 stages):**
 1. **Alignment:** use gravity (phone stationary) to fix pitch/roll, and acceleration direction while driving straight to fix yaw — done automatically, continuously re-checked.
 2. **AI Velocity Estimation:** a compact neural network reads a 1-second rolling window of motion data and outputs a single number — estimated forward speed

— trained on real driving data (IO-VNBD, 98 hours / 5,700 km across UK, France, and Nigeria).

3. **Sensor Fusion:** an Error-State Kalman Filter blends this AI speed estimate with GNSS (when available) and the physical rule that cars don't slide sideways, producing a continuous position estimate that degrades gracefully rather than catastrophically when GNSS drops out.
 4. **Map-Matching:** the raw trajectory is snapped onto the nearest plausible road using offline map data, correcting residual drift visually.
- **Working prototype plan:** offline algorithm validated on IO-VNBD first (no phone needed), then ported to a real Android app for live demo.

Diagram to include: the same 4-stage flowchart as Slide 2, but expanded with 1–2 words of detail under each box (e.g. under "AI Velocity Estimator": "1D-CNN / TCN, <2MB, on-device"), and a small inset showing GNSS-available vs. GNSS-denied as two parallel paths merging into the filter.

Slide 4 — Feasibility and Viability

Bullets — feasibility:

- Core AI task (window-of-IMU → single speed number) is a standard, well-documented supervised regression problem — tractable for a first-time ML team.
- Mandatory dataset is public, verified accessible, and already structured for this exact task (paired IMU windows + wheel-speed ground truth).
- Classical filter math is the highest-risk piece; mitigated by adapting existing open-source ESKF implementations rather than deriving from scratch.

Real risks + mitigations (pulled from source failure-mode analysis, not generic):

Risk	Mitigation
Android background battery-optimization (Doze Mode) silently drops IMU samples, breaking the AI model's input stream	Run sensor capture as a foreground service with a partial wake lock — a standard, documented Android pattern, not experimental.
Phone shifts position mid-drive (falls off mount, slides on seat) — alignment becomes wrong	Continuously monitor motion-signal variance; auto-recompute the alignment rotation the moment it looks physically implausible, without needing the user to notice or intervene.

Model trained on UK/France/Nigeria driving overfits to those road/traffic patterns and underperforms on Indian roads (speed breakers, mixed traffic, unpaved shoulders)

Collect a small (even 10–15 minute) local test drive before the final demo and validate the model against it — cheap insurance that also becomes a genuine talking point for judges.

GNSS reports a "confident" position that's actually wrong (multipath in urban canyons) — filter trusts bad data

Statistically reject GNSS updates that are wildly inconsistent with the AI-predicted trajectory before they're allowed to corrupt the filter.

Team underestimates the Kalman filter's math complexity and stalls

Pre-committed fallback: a simpler 2D linear filter (no NHC, no 3D attitude) that still demonstrates the core "AI keeps navigating when GPS drops" story, even if less accurate — always have something working on stage.

(Note: I did not find independent evidence of a "CPU thermal throttling" failure mode being unique to this problem — it's a generic mobile-ML concern rather than something specific to your architecture, so I've left it out in favor of the four above, which are architecture-specific and genuinely worth a judge's attention.)

Slide 5 — Impact and Benefits

Bullets:

- **Direct beneficiaries:** any Indian driver navigating through tunnels, underground parking, or dense urban canyons — situations where GPS is currently useless and drivers simply lose guidance.
- **Social:** improves safety and confidence for drivers in GPS-denied stretches without requiring any new hardware purchase — works on the phone people already own.
- **Economic:** a software-only solution is far cheaper to deploy at national scale than requiring OBD-II hardware or dedicated navigation units; directly usable by ride-hailing, logistics, and mapping companies.
- **Strategic/space-sector relevance:** strengthens India's homegrown navigation resilience — complements GNSS/NavIC rather than depending entirely on satellite availability, which matters for both civilian and strategic use cases ISRO cares about.
- **Environmental (secondary):** more reliable navigation reduces wrong-turn mileage and idling-while-lost, a small but real fuel-efficiency gain at scale.

No diagram required — this slide is bullet-only per most template conventions, but if space allows, a simple "before GPS loss / during GPS loss / after GPS loss" 3-panel comparison of a confused vs. confident driver UI works well here.

Slide 6 — Research and References

Only sources I independently verified as real and currently reachable:

- **Mandatory dataset:** Onyekpe, U., Palade, V., Kanarachos, S., & Szkolnik, A. (2021). *IO-VNBD: Inertial and Odometry Benchmark Dataset for Ground Vehicle Positioning*. Data in Brief, 35, 106885. Dataset: github.com/onyekpeu/IO-VNBD
- Onyekpe, U., et al. (2021). *WhONet: Wheel Odometry Neural Network for Vehicular Localisation in GNSS-Deprived Environments*. Engineering Applications of Artificial Intelligence, 105. Code: github.com/onyekpeu/WhONet
- Brossard, M., Barrau, A., & Bonnabel, S. (2019). *AI-IMU Dead-Reckoning*. IEEE Transactions on Intelligent Vehicles. Code: github.com/mbrossar/ai-imu-dr
- Liu, W., et al. (2020). *TLIO: Tight Learned Inertial Odometry*. (Cited as a related-but-not-directly-applicable technique — validated on pedestrian data only, not used as our architecture.)
- Valhalla open-source routing/map-matching engine: github.com/valhalla/valhalla
- Android developer documentation: foreground services & wake-lock best practices, developer.android.com/develop/background-work
- LiteRT (Android on-device inference runtime, successor to NNAPI-based deployment): ai.google.dev/edge/litert
- Smart India Hackathon 2026, Problem Statement 26168 — verify exact text at sih.gov.in before finalizing benchmark numbers used elsewhere in this deck.

(Two claims from your original source material I'm deliberately not citing here because I couldn't verify them independently: the 24-metric/100-point architecture scoring matrices in both reports, and the specific FOG-IMU-at-200Hz example — these read as illustrative reasoning by the report authors rather than citable facts, and your source material itself flagged the scoring matrices as unsourced opinion.)